

《迈向可解释的计算机自适应测试：基于大语言模型的方法》(LACAT) 精读笔记

原文: lacat-explainable-cat-with-llm_16.pdf(EMNLP 2024 Findings)·代码: <https://github.com/doublecheng12/LACAT>
(已克隆到本目录 LACAT/) ·笔记于 2026-06-27

一、解决什么问题

1. 背景：什么是 CAT

计算机自适应测试 (Computerized Adaptive Testing, CAT) 是「智能化的考试」：不像纸笔考试给所有人同一套题，CAT 会根据你前面答得怎么样，动态地给你出下一道题，目标是「用尽可能少的题，把你的真实能力估准」。

它有两个核心零件：- **认知诊断模型 (CDM)**：根据你的作答记录，给出一个能力估计值 θ (本文用最经典的 IRT, θ 是一个标量)。- **选择算法 (selection algorithm)**：根据当前的 θ ，从题库里挑下一道最该考的题。

流程是个循环：选题 → 学生作答 → CDM 更新 θ → 再选题……重复 T 步。

2. 痛点：现有方法「不会解释」

- **传统启发式** (MFI 最大 Fisher 信息、KLI)：靠固定数学规则选题，泛化差、建模不了复杂的师生交互。
- **深度学习/强化学习** (NCAT、BOBCAT、GMOCAT)：从海量作答数据里学一个选题策略，准是准了，但是黑箱——它为什么挑这道题？没人说得清。

教育场景里，老师和学生如果不知道「为什么给我出这道题」，就不信任这个系统。而已有工作几乎全在卷「用更少的题数」，没人做选题的可解释性。

3. 思路：用 LLM 当选题智能体

LLM 天然会「用人话讲道理」。如果让 LLM 来选题，它就能**边选边解释**。但直接把 LLM 塞进 CAT 有三个难点，本文用三个模块逐一对应解决：

难点	描述	对应模块
难点 1	CDM 只给一个能力向量 θ ，信息太少，不足以指导 LLM 选题	Summarizer ：把作答历史变成文字画像
难点 2	没有精确的「最优题」标准，LLM 还得解释自己的选择	Reasoner ：选题 + 给出人类可读理由
难点 3	缺反馈机制，过去的选题经验没被利用	Critic ：复盘过去选题，给改进提示

一句话总结贡献：**首个把 LLM 引入 CAT 做可解释选题的工作**，而且零样本（不训练）就能打平甚至超过需要训练的数据驱动方法。

二、问题定义（输入 / 输出）

任务：对学生 u_i ，在每一步 $t \in [1, T]$ ，从**支持集**里选一道题 q 让他答；答完后 CDM 更新能力估计 θ_t 。重复 T 次，目标让 θ_T 逼近真实能力 θ_0 。

符号约定：- 一条作答记录三元组： $e_{t'} = (q_{t'}, a_{t'}, c_{t'})$ ，其中 q 是题号、 $a \in \{0, 1\}$ 是对错、 c 是题面文本。- θ_t ：第 t 步 CDM 给出的能力估计 (IRT 下是标量)。- **支持集 (support set)**：可供选择的候选题 (占样本 80%)。- **查询集 (query set)**：留出来的题 (占 20%)，不拿来考，专门用来检验「当前估出的 θ_t 准不准」——拿 θ_t 去预测学生在查询集上的对错，和真实对错比，算 ACC/AUC。

一个具体小例子 (贴近 KT/CAT 场景)：

学生小明，题库有 100 道初中数学题。随机划分：支持集 80 道、查询集 20 道。- 第 1 步：系统先给一道「区分度高、难度中等」的代表题，比如一道一元二次方程题。小明答对 $\rightarrow e_1 = (q_{37}, 1, \text{“解方程 } x^2 - 5x + 6 = 0\text{”})$ 。- CDM 更新得 $\theta_1 = 0.3$ 。用 θ_1 去预测小明在 20 道查询题上的对错，算出 ACC=0.65。- 第 2 步：Summarizer 看了 e_1 ，写出画像「该生代数较强」；Reasoner 据此从支持集挑下一题并解释「选这道几何题来探测他的弱项」…… - 如此循环到第 T 步。

最终输出：每一步给出 (r_t, q_t) ——选的题 q_t 和选它的自然语言理由 r_t 。这个 r_t 就是本文相对传统 CAT 多出来的「可解释性」。

三、模型结构

整体是 **Summarizer** $\rightarrow$ **Reasoner** $\leftarrow$ **Critic** 三模块循环，全部由 LLM (GPT-3.5-turbo-16k, 温度=0) 驱动，不训练任何参数。

模块 1: Summarizer (总结器) ——把数字变成画像

负责把「干巴巴的作答记录 + 一个 θ 值」翻译成「人能读、LLM 能用」的学生画像。分两层：

- 长期画像 L_t ：吃进全部历史记录 $[e_1, \dots, e_{t-1}]$ 和当前 θ ，总结出学生在各知识点上稳定的强项/弱项。比如「擅长代数，几何（尤其立体几何）持续薄弱」。对应公式 (1)。
- 短期画像 S_t ：只看最近一次作答 e_{t-1} ，结合长期画像 L_t ，产出一条「指导性想法」。比如「他刚答对了三角函数题，下一题这个知识点可以加难度，以更精确地探测」。对应公式 (2)。

为什么分长/短期？因为能力既有长期稳定性（空间想象差的人立体几何一直差），也有即时动态（刚答对就该加难度）。这其实呼应了 KT 里 long-term/short-term perception 的思路。

模块 2: Critic (评判器) ——复盘纠错

它先判断上一轮选题质量好不好：质量的定义是「这道题有没有让能力估得更准」，即查询集准确率有没有上升（公式 3）。如果质量低（准确率没升），Critic 就用 LLM 分析「为啥这次选砸了」，产出一条提示 C_t （公式 4）。

典型诊断（论文表 1）：「选的三角函数题对学生太难了 → 下次该降低这个知识点的难度」。这条提示会喂给 Reasoner，避免重复犯错。

模块 3: Reasoner (推理器) ——选题 + 解释

真正做选择的模块。它综合三路信息：长期画像 L_{t-1} 、短期画像 S_{t-1} 、Critic 提示 C_{t-1} ，外加候选题 (support set)，用思维链 (CoT) 提示，从难度、区分度、知识覆盖三个角度权衡，最终输出选哪道题 + 为什么选（公式 5）。

典型输出（论文表 2）：「我选第 1 题，因为它考一元二次方程，学生近期在这块进步明显，中等难度能有效评估并区分其代数能力；同时避开他薄弱的几何，让他专注强项。」

数据流 (一步内的循环)

```
历史记录 e_1..e_{t-1}, theta
|
v
[Summarizer] --> L_t (长期画像), S_t (短期画像)
|
v
[Reasoner] (吃 L,S,C + 候选题) --> (r_t, q_t) 选题 + 理由
|
v
学生作答 --> CDM 更新 theta_t --> 算查询集 ACC
|
v
[Critic] (看 ACC 升没升) --> C_t 提示
|
+----- 回灌给下一轮的 Reasoner (闭环)
```

冷启动：新学生没记录时，先发一道「高区分度、中难度」的代表题打底。

四、关键公式详解

公式 1: 长期画像

$$L_t = \text{LLM}_{\text{long-term}}([e_1, e_2, \dots, e_{t-1}], \theta_{t-1}, \text{prompt}_{\text{long-term}})$$

- 符号含义： $[e_1, \dots, e_{t-1}]$ 是到目前为止全部作答三元组（题、对错、题面）； θ_{t-1} 是上一步的能力估计（IRT 下是一个标量）； $\text{prompt}_{\text{long-term}}$ 是固定的提示模板（见译文附录 F）；输出 L_t 是一段自然语言（强项 + 弱项）。
- 举例：假设 $t = 6$ ，前 5 题里学生答对了 3 道代数题 $\{e_1, e_3, e_5\}$ 、答错了 2 道几何题 $\{e_2, e_4\}$ ， $\theta_5 = 0.4$ 。把「Correct: [代数题 1,3,5], Incorrect: [几何题 2,4], Ability: 0.4」填进模板喂给 LLM，输出 $L_6 =$ 「**Strength**: 一元二次方程、因式分解掌握扎实；**Weakness**: 平面几何中的角度计算反复出错」。
- 作用：把稀疏的数字记录压缩成结构化的文字理解，解决难点 1（单靠 θ 向量信息不够）。它是后续选题的「知识底座」。

公式 2：短期画像

$$S_t = \text{LLM}_{\text{short-term}}([e_{t-1}], L_t, \text{prompt}_{\text{short-term}})$$

- 符号含义：只输入最近一条记录 e_{t-1} 和刚算出的长期画像 L_t ；输出 S_t 是一条「想法 (thought)」。
- 举例：接上例，第 5 题 e_5 是「三角函数 $\sin$ 值计算，答对」。把它和 L_6 一起喂模型，得 $S_6 = \text{「Thought: 学生刚正确处理了三角函数基础题，说明该知识点已入门，下一题可在三角函数上适当提高难度，以更精确地探测其上限。」}$ 。
- 作用：捕捉即时动态。长期画像偏稳态，短期画像负责「趁热打铁」地调难度，让选题更贴近当前状态。

公式 3：题目质量奖励 r_{qua} (Critic 的核心信号)

$$r_{\text{qua}} = \begin{cases} 1, & \text{若 } \text{ACC}(\theta_t) > \text{ACC}(\theta_{t-1}) \\ 0, & \text{否则} \end{cases}$$

- 符号含义： $\text{ACC}(\theta_t)$ 是用第 t 步估出的能力 θ_t 去预测查询集作答的准确率。这是个二值奖励：这一题答完后，能力估得更准了 (ACC 上升) 就给 1，否则给 0。
- 为什么能这么定义：真实能力 θ_0 不可知，没法直接判断估计准不准。但查询集是「留出来的标准答案」， θ_t 预测查询集越准，说明 θ_t 越接近真值。所以查询集 ACC 的上升就是「这道题选得好」的代理信号。
- 举例：第 4 步前 $\text{ACC}(\theta_3) = 0.70$ ；选了一道题、学生答完、CDM 更新到 θ_4 ，重算 $\text{ACC}(\theta_4) = 0.73 > 0.70 \rightarrow r_{\text{qua}} = 1$ (这道题选得好)。若变成 $0.68 \rightarrow r_{\text{qua}} = 0$ (选砸了，触发 Critic 反思)。
- 作用：给 Critic 一个客观的好坏判据，解决难点 3 (缺反馈机制)。

【注意】论文 vs 代码的差异 (重要)：公式 3 写的是「ACC 上升」。但代码 `src/environment/env.py` 里 `step()` 实际用的是 AUC 上升做二值奖励 (`auc_add = 1 if auc_add > 0 else 0`)，并且把奖励组成了一个三元组 `reward = [auc_add, rate_change, cov]`：- `auc_add`：查询集 AUC 是否提升 (对应公式 3 的精神，但用 AUC 而非 ACC)；- `rate_change`：所选题与已考题是否「新颖」(用题面 embedding 余弦相似度 > 0.85 判定相似，见 `check_similar_question`)；- `cov` (`compute_div_reward`)：这道题是否引入了新知识点 (知识覆盖多样性)。

也就是说实现里把「质量 + 新颖度 + 知识覆盖」三者合并，比论文正文里单一的 r_{qua} 更丰富——这正好解释了为什么 LACAT 在 RQ3 里曝光率/重叠率 (novelty) 那么低。

公式 4：Critic 提示生成

$$C_t = \text{LLM}_{\text{critic}}([e_1, \dots, e_{t-1}], \theta_{t-1}, \text{prompt}_{\text{critic}})$$

- 符号含义：当 $r_{\text{qua}} = 0$ (选砸了) 时触发，输入历史记录和能力估计，输出一条诊断提示 C_t 。
- 举例：上一题给学生出了道超纲的三角函数题，ACC 没升。Critic 输出 $C_t = \text{「Hint: 上一题难度超出学生能力，导致挫败；接下来应降低该知识点难度，循序渐进。」}$ 。
- 作用：把「错在哪、下次怎么改」用文字传给 Reasoner，形成闭环纠错。代码对应 `src/llms/critic.py`，提示模板内置了三类典型错误 (太难/太易/知识饱和) 供 LLM 对照诊断。

公式 5：Reasoner 选题

$$(r_t, q_t) = \text{LLM}_{\text{Reasoner}}(L_{t-1}, S_{t-1}, C_{t-1}, [\text{support set}], \text{prompt}_{\text{Reasoner}})$$

- 符号含义：输入长期画像、短期画像、Critic 提示、候选题集；输出理由 r_t 和选中的题 q_t 。注意输出里既有 q_t (给 CAT 用) 也有 r_t (给人看) ——可解释性就体现在这。
- 举例：把 $L = \text{「代数强、几何弱」}$ 、 $S = \text{「三角函数可加难度」}$ 、 $C = \text{「别再出超纲题」}$ 、候选题列表喂给模型，输出：Reason: 选索引 3 的中等难度三角函数题，因为它能在学生已入门的知识点上精确探测能力上限，且难度适中不会挫败；Selected question with index: 3。
- 作用：核心决策器，解决难点 2 (既要选好题又要会解释)。CoT 提示让它「先分析强弱项 → 再按难度/区分度/知识覆盖权衡 → 给理由」。代码对应 `src/llms/generator.py` 的 `cog_answer` 与 `src/cat/cat_agent.py`。

五、代码实现对照

克隆仓库 LACAT/ 结构与论文公式的对应：

论文概念/公式	代码位置	说明
公式 1 长期画像 L_t	src/llms/longterm_memory.py → LongTermMemory.answer(right_ids, strength/weakness, wrong_ids)	用 LONGTERM_MEMORY_PROMPT 填入对/错记录, LLM 输出
公式 2 短期画像 S_t	src/llms/short_memory.py → ShortTermMemory.answer(profile, thought, last_question)	用 SHORT_MEMORY_PROMPT, 输出
公式 3 质量奖励	src/environment/env.py → step() / reward()	实际用 AUC 提升, 且合并 [auc_add, rate_change, cov] 三元奖励
公式 4 Critic C_t	src/llms/critic.py → Critic.answer(profile, last_question)	用 CRITIC_PROMPT, 输出 feedback
公式 5 Reasoner	src/llms/generator.py、 src/cat/cat_agent.py → CATAgent.run() 调 generator.cog_answer(...)	综合 strength/weakness/last_question/candidate/knowledge 选题
CDM (IRT)	src/models/irt_model.py	adaptest_update 更新 θ , evaluate 算查询集 acc/auc; get_theta(sid) 取能力
提示模板	src/prompts/prompt_templates.py	四个模块的 prompt (与译文附录 F 对应)
题面 embedding/相似度	src/utils/data_utils.py get_embedding + env.check_similar_question	余弦相似度 >0.85 判相似, 服务于新颖度奖励
主流程	main.py	串起环境、CDM、三模块的 T 步循环

几个值得实现的细节: - LLM 输出用 eval(cleaned_output) 解析 (先正则去掉“json 包裹”), 解析失败就回退原文——比较脆, 依赖 LLM 严格按格式输出。 - 数据集默认 moocradar, CDM 默认 IRT, checkpoint 写在 ckpt/moocradar/irt.pt。 - 支持/查询集划分在 env.split_data(ratio=0.8)’, 与论文 80%/20% 一致。

六、小结

实验结论: - **RQ1 (准确率):** 在 Junyi、MoocRadar 上, 零样本的 LACAT 全面超过需要训练的 BOBCAT/NCAT (如 MoocRadar AUC@10: LACAT 76.32 vs NCAT 76.01)。但在 **Exam** 数据集 (初中数学, 需复杂推理) 上偏弱——LLM 看不懂难数学题的属性。 - **RQ2 (可解释性):** 人工评估 Fleiss’Kappa 普遍 >0.41 (流畅度近 1.0 几乎完全一致), 平均分流畅度近 2.0、相关/全面 >1.5; 自动评估 (GPT-4 + Deepseek-v2) >90% 内容有效。说明解释质量确实高。 - **RQ3 (新颖度):** LACAT 文本重叠率最低 (3.69%), 曝光率也很低 → 不会反复出相似题。 - **RQ4 (消融):** 去掉 Summarizer (-S) 掉得最多 (AUC@10 76.32→74.20), 去掉 Critic (-C) 也降 (→75.17), 证明两个模块都有用, Summarizer 更关键。

优点: 1. 首个做 CAT 可解释性的工作, 每次选题都给人话理由, 提升师生信任。2. 零训练就能打平/超过数据驱动方法, 迁移成本低。3. 三模块设计干净, 分别对应三个难点。

局限: 1. 只用了单维 IRT 的标量 θ 。多维 CDM (NCDM、RCD 等) 的能力向量如何转成文本喂给 LLM, 仍是难题。2. 复杂推理题 (Exam) 上 LLM 力不从心, 性能依赖底座模型的推理力 (Deepseek-v2 > GPT-3.5 > Qwen-1.5-7B)。3. 可解释性评估指标是本文首创, 尚不成熟, 需进一步定义。4. 效率: 每步选题约几秒 (DeepSeek ~8s), 靠「并发预选」缓解; 成本可控 (最贵每轮 <\$0.2)。

对 KT/CAT 研究者的启发: 这套「LLM 把数字画像化 + 自带解释 + 反思纠错」的思路, 可以迁移到知识追踪的可解释性、以及多维认知诊断向量的文本化表达上——后者正是论文自己点名的未来方向。